

Pflichtenheft: Gastro-Reservierungs-System

1. Projektziele und Ausgangssituation

1.1. Ist-Zustand Gegenwärtig verwalten viele lokale Gastronomiebetriebe ihre Tischreservierungen manuell über physische Reservierungsbücher. Dies führt häufig zu organisatorischen Problemen wie Doppelbuchungen, unübersichtlicher Auslastung, verlorenen Kundendaten und einem hohen zeitlichen Aufwand für das Personal bei der telefonischen Platzsuche.

1.2. Soll-Zustand (Projektziel) Es soll ein digitales Backend-System zur Verwaltung von Tischen, Kunden und Reservierungen entwickelt werden. Das System automatisiert die Überprüfung der Verfügbarkeiten, verhindert Fehlbuchungen und bietet jederzeit einen klaren Überblick über die Auslastung. Ziel ist es, eine stabile und persistente Datenbasis zu schaffen, die später als Grundlage für eine grafische Benutzeroberfläche dienen kann.

2. Funktionale Anforderungen (Was das System tun muss)

2.1. Muss-Kriterien (Kernfunktionen)

- **Stammdatenverwaltung Tische:** Das System muss es ermöglichen, neue Tische mit einer eindeutigen Nummer, der Anzahl der Sitzplätze und dem Standort (z. B. "Innenbereich", "Terrasse") anzulegen und abzurufen.
- **Stammdatenverwaltung Kunden:** Kunden müssen mit Namen und einer Kontaktmöglichkeit (z. B. Telefonnummer) im System registriert und ausgelesen werden können.
- **Reservierungslogik:**
 - Das System muss Reservierungen für ein bestimmtes Datum und eine bestimmte Uhrzeit erstellen können.
 - Dabei wird ein Kunde mit einem bestimmten Tisch verknüpft.
- **Konfliktprüfung:** Vor dem Speichern einer Reservierung muss das System zwingend prüfen, ob der ausgewählte Tisch zur gewünschten Zeit bereits belegt ist. Ist dies der Fall, muss die Buchung mit einer Fehlermeldung abgelehnt werden.
- **Übersichten:** Es muss möglich sein, alle getätigten Reservierungen sowie alle registrierten Kunden und Tische aufzulisten.

2.2. Kann-Kriterien (Optionale Erweiterungen für die Zukunft)

- Stornierung von bestehenden Reservierungen.
- Zuweisung von Reservierungen nach Personenanzahl (Das System schlägt automatisch einen Tisch vor, der groß genug für die Gruppe ist).
- Definition von Zeitfenstern (z. B. eine Reservierung blockiert den Tisch standardmäßig für 2 Stunden).

3. Nicht-funktionale Anforderungen (Wie das System arbeiten muss)

- **Programmiersprache:** Java (JDK 14).
- **Datenhaltung:** Relationale Datenbank (SQLite), um eine unkomplizierte, lokale Speicherung ohne externen Serverbetrieb zu gewährleisten.
- **Datenzugriff (Persistenz):** Die Anbindung an die Datenbank erfolgt zwingend objektrelational mittels des Frameworks **ORMLite (Version 6.1)**.
- **Softwarearchitektur:** Das System muss streng nach dem Schichtenmodell getrennt sein. Datenbankzugriffe erfolgen über das **DAO-Muster** (Data Access Object). Die Geschäftslogik (z.B. Konfliktprüfung) wird in einer separaten **Service-Schicht** gekapselt.
- **Benutzerschnittstelle:** Für den Prototyp wird eine textbasierte Konsolenanwendung (CLI) via Scanner implementiert, die den Benutzer über ein strukturiertes Menü durch die Anwendungsfälle führt.
- **Robustheit:** Das System muss Fehleingaben (z. B. Buchstaben statt Zahlen bei IDs) abfangen und darf dabei nicht abstürzen.

4. Datenmodell (Entitäten)

Das System basiert auf drei zentralen Entitäten, die in der Datenbank abgebildet werden:

Tisch

- id (Primary Key, Auto-Increment)
- tischNummer (Integer, Unique)
- anzahlSitzplaetze (Integer)
- bereich (String)

Kunde

- id (Primary Key, Auto-Increment)
- name (String)

- telefonnummer (String)

Reservierung

- id (Primary Key, Auto-Increment)
- tisch_id (Foreign Key auf Tisch)
- kunde_id (Foreign Key auf Kunde)
- datum (String, Format: YYYY-MM-DD)
- uhrzeit (String, Format: HH:MM)

<https://github.com/dogukansentrk/GastroReservierung.git>